

CS 630: Software Testing and Quality Assurance

1-3 Activity: Navigating the Console

Malgorzata Debska

Southern New Hampshire University

Dr. Ben Torrales

July 9, 2026

Exploratory Testing Report

Software testing is a fundamental component of the software development lifecycle because it helps ensure that applications function correctly, meet user expectations, and provide a reliable user experience. When formal test cases are unavailable, exploratory testing offers an effective approach for evaluating software by allowing testers to interact with the application while simultaneously learning how it behaves. This method encourages critical thinking, flexibility, and observation, making it well suited for identifying functional defects, usability concerns, and workflow limitations that may not be captured through scripted testing alone.

For this activity, I performed exploratory testing on a console-based C++ application designed to convert Fahrenheit temperatures into Celsius. The application reads temperature data from an input file, performs the necessary calculations, and writes the converted values to a separate output file. Because no predefined test plan was provided, I developed my own testing strategy based on program documentation, source code, and expected functionality. The objective was to verify that the application performed its intended task correctly while also evaluating its usability and identifying opportunities for improvement.

Exploratory Testing Approach

Before executing the application, I reviewed the project documentation, source code, compile script, and sample input file to gain an understanding of how the application was intended to function. I then compiled the application using the provided shell script for macOS/Linux and executed the resulting program through the command-line interface.

During testing, I observed the application's behavior throughout the entire workflow, beginning with compilation and ending with the generation of the output file. After the program completed execution, I verified that the output file had been created and examined its contents to ensure the temperature conversions were performed correctly.

The application used the provided input file, **FahrenheitList.txt**, which contained the following sample values:

- Braddock
- Frederick

The application successfully converted the temperatures associated with these entries and generated the following output:

- Braddock 0
- Frederick 25

In addition to confirming successful execution, I evaluated the application's usability by examining the clarity of its console output, the effectiveness of its feedback messages, and the overall workflow experienced by a first-time user.

Test Results

Test Interaction 1: Compiling the Application

The first step involved compiling the source code using the provided shell script. Successful compilation was necessary before the application could be executed.

Expected Result

The application should compile without errors and generate an executable program.

Actual Result

The compilation completed successfully and produced the executable file CS630-1-3. No compilation errors or warnings were encountered.

Result: Pass

Test Interaction 2: Executing the Program

After compilation, I executed the program through the Terminal using the generated executable.

Expected Result

The application should read the input file, convert the Fahrenheit temperatures to Celsius, create an output file, and terminate successfully.

Actual Result

The application completed execution successfully and displayed the message:

Done.

No runtime errors or crashes occurred during execution.

Result: Pass

Test Interaction 3: Verifying Output File Creation

Following program execution, I verified whether the application generated the expected output file.

Expected Result

The application should create a file named **CelsiusList.txt** containing the converted temperature values.

Actual Result

The output file was successfully created in the project directory.

Result: Pass

Test Interaction 4: Validating Output

The final step involved reviewing the contents of the generated output file to verify that the conversions were completed correctly.

Expected Result

The output file should contain the converted Celsius values corresponding to the temperatures stored in the input file.

Actual Result

The output file contained the following data:

- Braddock 0
- Frederick 25

The generated values were consistent with the expected functionality of the application.

Result: Pass

Issues Identified During Testing

Although the application completed all required tasks successfully, exploratory testing identified several areas where usability and maintainability could be improved.

Issue 1: Minimal User Feedback

Priority: Low

The application displays only the message "**Done.**" After processing is complete. While this indicates successful execution, it does not provide users with meaningful information about what occurred during processing.

For example, the application does not indicate:

- which input file was processed,
- whether the conversion completed successfully,
- where the output file was saved,
- or how many temperature records were converted.

Providing more descriptive completion messages would improve the overall user experience and reduce uncertainty for users who may not be familiar with the application.

Issue 2: Limited Error Handling

Priority: Medium

During successful execution, no errors occurred because the supplied dataset contained valid input. However, based on the observed workflow and source code, the application appears to rely on correctly formatted input without providing extensive validation or descriptive error

reporting. If the input file were missing, improperly named, or contained invalid data, users would likely receive minimal information regarding the cause of the problem. Implementing stronger exception handling and user-friendly error messages would improve both software robustness and maintainability.

Issue 3: Workflow Improvements

Priority: Low

The overall workflow is straightforward but lacks interaction with the user. The application begins processing immediately and exits once the conversion is complete. Although this design is appropriate for a simple console application, displaying progress information or a summary of the completed operation would improve usability. For example, informing the user that two temperature records were processed successfully would provide greater confidence that the application performed as expected.

Comparison of Expected and Actual Results

Test Activity	Expected Result	Actual Result	Outcome
Compile application	Executable generated successfully	Executable CS630-1-3 created successfully	Pass
Execute application	Program completes without errors	The program was executed successfully and displayed "Done."	Pass
Generate output file	Create CelsiusList.txt	Output file created successfully	Pass
Verify converted data	Correct Celsius values generated	Output contained correct converted temperatures	Pass

To conclude, the exploratory testing process demonstrated that the application successfully fulfilled its primary functional requirements. The software compiled without errors, executed successfully, processed the supplied input file, generated the expected output file, and produced accurate temperature conversions. No functional defects were observed while testing the provided dataset. Although the application's core functionality performed as expected, exploratory testing identified several opportunities to improve the overall user experience. Enhancements such as clearer status messages, stronger input validation, and more informative error handling would make the application easier to use and troubleshoot while increasing its reliability when processing unexpected input.

This activity illustrates the value of exploratory testing within software quality assurance. Even when software functions correctly, exploratory testing can reveal usability concerns and opportunities for refinement that contribute to a more robust and user-friendly application. By combining technical verification with user-focused evaluation, exploratory testing provides valuable feedback that supports continuous software improvement throughout the development lifecycle.